The Journal of Systems and Software 237 (2026) 112857

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Automated assessment of the relationship between microservice architectures and performance

Alberto Avritzer [a], Andrea Janes [b,*], Helena Rodrigues [c], Yuanfang Cai [d], Teiji Schoyen [d], Ernst Pisch [d], Catia Trubiani [e], Andre B. Bondi [f], Daniel Sadoc Menasché [g], Chenxi Zhang [h]

[a] eSulabSolutions Inc., Princeton, NJ, USA
[b] Free University of Bozen-Bolzano, Bolzano, Italy
[c] LASI/Centro Algoritmi, Braga, Portugal
[d] Drexel University, Philadelphia, USA
[e] Gran Sasso Science Institute, L'Aquila, Italy
[f] Software Performance and Scalability Consulting LLC, Red Bank, NJ, USA
[g] UFRJ, Rio de Janeiro, Brazil
[h] Xidian University, Xi'an, China

ARTICLE INFO

Dr T. Mårtensson

Keywords:
Microservice architecture
Performance evaluation
Antipattern detection

ABSTRACT

A microservice architecture is intended to promote modularity and evolvability. In this paper, we present an automated framework for assessing the relationship between microservice architecture complexity and performance-related quality attributes. In this framework, we use PPTAM, a performance testing tool, to evaluate system response time under varying user loads, and DV8, an architecture analysis tool, to assess architectural complexity and the complexity of individual services using coupling scores, propagation cost, and architectural antipatterns derived from various types of dependency relations. Using this approach, we evaluated five benchmark systems, including four releases of a microservice system that share similar functionalities but differ in structural design. The results show that microservice architectures with poor complexity scores also exhibited degraded performance outcomes. This automated framework, for the first time, enables a comprehensive measurement of microservice architecture complexity, formed through multiple types of statically extracted dependencies, and its correlation with dynamically obtained performance metrics.

1. Introduction

Microservice architecture (MSA) has emerged as a prominent design paradigm for the decomposition of large monolithic systems into collections of small, autonomous, and independently deployable services. The design process involves a wide range of decisions concerning data distribution, communication protocols, service orchestration, and deployment configurations. Although microservices are widely believed to promote modularity and evolvability, the impact of their architectural complexity, formed by various types of static dependencies, on runtime quality attributes, such as performance and scalability, is not sufficiently understood. Assessing performance in microservice systems is inherently dynamic and depends on runtime environments and workloads. In contrast, design structures are static, formed by various dependencies connecting distributed entities. The research question is: *"What is the relationship between architectural complexity and performance outcomes in microservice systems?"*

To answer this question, in this paper, we contribute a fully automated framework that combines static architecture analysis with dynamic performance testing to detect the relationships between structural complexity and performance degradation. As shown in Fig. 1, our framework integrates: (1) **PPTAM** (Avritzer et al., 2018), a performance testing tool that records response time (performance) and scalability metrics (supported number of users) under varying load condition; and (2) **DV8** (Cai and Kazman, 2019), an architecture analysis tool that measures design complexity using various metrics and antipatterns. The input of DV8 is a Design Structure Matrix (DSM), which we constructed from different types of dependencies extracted from the code repository.

* Corresponding author.
E-mail addresses: beto@esulabsolutions.com (A. Avritzer), ajanes@unibz.it (A. Janes), helena@dsi.uminho.pt (H. Rodrigues), yc349@drexel.edu (Y. Cai), tjs392@drexel.edu (T. Schoyen), epp26@drexel.edu (E. Pisch), catia.trubiani@gssi.it (C. Trubiani), andrebbondi@gmail.com (A.B. Bondi), sadoc@dcc.ufrj.br (D.S. Menasché), zhangchenxi@xidian.edu.cn (C. Zhang).

https://doi.org/10.1016/j.jss.2026.112857
Received 15 October 2025; Received in revised form 27 January 2026; Accepted 12 March 2026
Available online 20 March 2026
0164-1212/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

The quantitative outputs from these tools enable us to assess the relationship between architecture complexity, performance and scalability outcomes systematically. In this paper we evaluate system scalability by measuring system performance under varying load levels.

This work extends (Avritzer et al., 2025b), in which we manually constructed the DSM and analyzed architectural complexity and performance metrics for a single benchmark system. Based on these promising results, we first replaced manual DSM construction with a fully automated end-to-end framework, covering dependency extraction, DSM construction, complexity metric computation, and performance analysis. Second, we expanded the evaluation from a single benchmark to five microservice benchmarks, enabling a more robust assessment of the relationship between architectural complexity and performance. Our extended study identified a strong correlation between the coupling score of individual services and degraded performance, and a clear trend that the microservice systems with poorer complexity scores and higher Clique ratios exhibited lower performance overall. The key contributions of this work are:

- We introduced a fully automated framework that quantifies the relationship between microservice architecture complexity, derived from multiple types of statically detected dependencies, and performance-related quality attributes.
- We analyzed five complex benchmark systems, including four structurally distinct releases of the Train-Ticket microservice benchmark (Zhou et al., 2018) and one instance of the DeathStarBench (University, 2019) suite. Train-ticket has been used extensively in the microservice research community to evaluate architecture and performance (e.g., in Fischer et al., 2024; Giamattei et al., 2024; Camilli et al., 2022a). The complexity of Train-ticket is illustrated in the fan-in/fan-out Fig. 3. A recent literature review based analysis of 378 Microservices-based Open-Source Projects shows that the number of microservices in such projects ranges from 3 to 50 (d'Aragona et al., 2024). Train-ticket has 42 micro-services and was ranked third in this analysis.
- Regarding the relationship between microservice-architecture complexity and overall performance, our analysis demonstrated that systems with poor structural complexity, reflected in high propagation cost —when one component changes, how large a proportion of the other components is likely to be affected, and high Clique ratios The proportion of components whose dependencies form a strongly connected graph, exhibited reduced performance and supported fewer user requests.
- To assess the relationship between individual service complexity and their performance, we applied Spearman's rank correlation to assess the correlation between each endpoint's coupling score and its performance score. The results yielded a correlation coefficient of 0.6781 with a p-value of 1.19×10^{-7}, indicating a statistically significant positive correlation between the two: service endpoints with higher coupling scores tend to have longer response times.

This work makes the first automated connection between statically extracted architectural structures and dynamically measured performance outcomes in microservice systems. It opens new research directions in understanding the interplay between structural design and runtime behavior. While our current framework focuses on two dependency types (API calls and shared data), future work will explore other forms such as configuration and build-time dependencies. We also plan to extend the framework to support trade-off analysis across multiple quality attributes, including resource consumption, maintainability, and availability.

This paper is structured as follows. Section 2 presents an overview of the related work. Section 3 introduces the benchmarks used for performance and architecture assessment. Section 4 describes the automated performance and architecture assessment framework. Section 5 presents the empirical study using the automated assessment framework, while Section 6 presents the performance and architecture analysis results. Section 7 presents the correlation analysis. Section 8 presents the identified threats to validity. Finally, Section 9 concludes and discusses future research.

2. Related work

Related work on *architecture complexity*, and *performance analysis* tends to address these two domains without considering the interdependence between them (Cai et al., 2025a; Avritzer et al., 2021). In contrast, this work relates to approaches conceived to understand the relationships between architectural abstractions and system performance glitches. Recent literature studies (Rodrigues et al., 2025; Li et al., 2021) confirm the importance of understanding and supporting the improvement of quality attributes in microservices architectures. In this domain, Liao et al. (2025) represent a very recent research effort that identifies performance regressions by bridging component-level testing with system-level architectural models. We share the intention of detecting performance issues in the early stages of software development, but our approach focuses on assessing microservice architecture complexity, derived from static analysis, and performance outcomes that are measured dynamically at runtime.

Next, we categorize the most relevant related work into five areas: architecture bad smells and performance, correlation metrics, performance antipatterns, automated performance testing frameworks focusing on microservice architectures, and microservice and monolith architectures.

Architecture bad smells and performance. Liu et al. (2021) present an empirical study that provides evidence of the impact of architectural bad smells in the performance of microservice systems. This is achieved by injecting three selected smells and measuring their performance slowdown. Engel et al. (2018) analyze five microservice-based projects and derive their architectural models, which in turn trigger the definition of six metrics, such as the number of synchronous cycles. The goal is to identify hotspots, i.e., specific parts of the architecture where flaws are more likely to occur.

Correlation metrics. Cai et al. (2025a) collect three categories of metrics: (i) architectural complexity, measured using propagation cost (MacCormack et al., 2006), decoupling level (Mo et al., 2016), and structural antipatterns (Mo et al., 2021); (ii) maintenance activity, measured using the number of changes, lines of code written, and active coding time spent on feature-addition vs. bug-fixing, and (iii) developer sentiment on complexity and productivity. Gu et al. (2025) point out the difficulty of detecting diverse and evolving anomalies between different services, and propose a framework to address this problem. The novelty consists in understanding whether predicted anomalies have an actual correlation with business exceptions. Bakhtin et al. (2025) focus on microservice architectures and investigate the metrics used in the context of network centrality (e.g., closeness and betweenness) in correlation with quality metrics that are expressed at a high level of generality (e.g., code smells and security rating).

Our work leverages the key findings of these prior publications (Liu et al., 2021; Engel et al., 2018; Cai et al., 2025a; Gu et al., 2025; Bakhtin et al., 2025) while contributing an automated framework to assess the relationship between architecture complexity and performance outcomes.

Performance antipatterns. Several approaches have been defined in the literature to identify software performance antipatterns, since they represent a valuable source of knowledge to bridge architectural choices and system performance issues. The detection of software performance antipatterns may target different systems' abstractions, specifically: (i) architectural models, e.g., traceability links between architectural models and model-based performance analysis results are derived in Trubiani et al. (2017); (ii) performance testing results, e.g., a methodology including statistical characterization of antipatterns and a multivariate analysis based on performance testing results has been proved effective on a large real system in Avritzer et al. (2021), Avritzer et al. (2022);

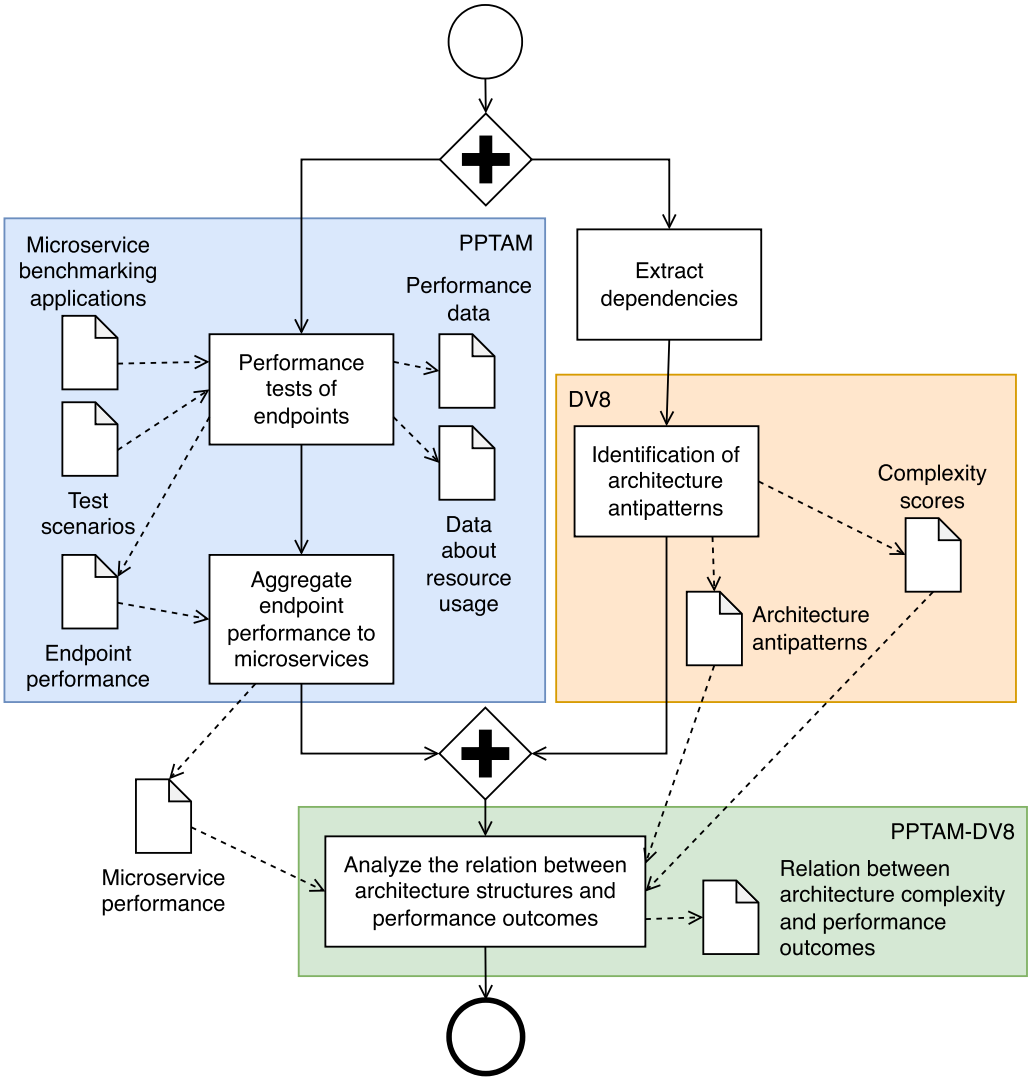


Fig. 1. Automated Microservice Architecture Complexity and Performance Analysis Framework.

(iii) source code, e.g., Java applications can be profiled, and both static and dynamic analysis of the code contribute to identifying antipatterns, as shown in Trubiani et al. (2023). Our work gets inspiration from these prior publications (Trubiani et al., 2017; Avritzer et al., 2021, 2022; Trubiani et al., 2023), but our focus is on the relationship between statically measured architecture complexity and dynamically measured quantitative performance outcomes, rather than performance antipatterns.

Automated performance testing frameworks. The attention to microservices architectures has been triggered by major players, such as Netflix, Amazon, and Uber (Rud, 2019), since they decided to migrate to microservices because of their promise of being scalable, efficient, and fault-tolerant (Larrucea et al., 2018). In the following, we report on automated testing frameworks, highlighting the main differences between their approach and ours. Detti et al. (2023) propose a tool to benchmark cloud/edge computing platforms and run microservice applications. One of the key benefits is the ability to vary configuration parameters, such as the number of microservices involved and the resources used. Quattrocchi et al. (2024) also target cloud applications and test autoscaling solutions to optimize the usage of available resources, thereby improving the system performance. Liang et al. (2026) recently present an autoscaling technique tailored for microservice applications. Specifically, a spatio-temporal graph neural network model is defined to capture time-varying dependencies between services and improve the accuracy of resource demand predictions. De Camargo et al. (2016) present an architecture to automate performance tests on microservices systems. However, the automated execution of performance tests in microservice architectures requires application changes. This is invasive. Avritzer et al. (2018) propose a quantitative approach to the assessment of microservice architecture deployment alternatives by automated performance testing. The approach is complemented by a tool that (i) defines a scalability requirement for each microservice, (ii) computes the domain metric as a normalized performance metric based on the probability of execution; (iii) creates a polygon that depicts the degradation in the domain metric as a function of a varying load level, due to failed performance tests. Camilli et al. (2022b) extend the tool using a formal description of user behavior through a Discrete-time Markov Chain (DTMC). Wang et al. (2019) analyze microservices interfaces to create an automated testing framework. Raychev (2020) discuss performance testing automation methodologies in the context of migrating from monolithic applications to microservice architecture.

Our work makes use of the insights learned from Detti et al. (2023), De Camargo et al. (2016), Avritzer et al. (2018), Camilli et al. (2022b), Wang et al. (2019), Raychev (2020), Blinowski et al. (2022) since performance testing is a crucial operational stage in our empirical assessment from which we collect performance outcomes, but our focus in this paper is the automated detection of the correlation between architecture complexity and performance regression.

Microservice and Monolith Architectures. Blinowski et al. (2022) investigate the differences between monolithic and microservice architectures, and experiments show that scaling out beyond a certain number of instances degrades application performance. Berry et al. (2024) question when it is worth migrating a monolith application to microservices, and an experience report is provided to identify the pros and cons, not only in terms of system performance, but also in relation to other quality attributes, such as availability and energy usage. Bosilia et al. (2025) compare the resilience and robustness of microservice architectures with those of their equivalent monolithic counterparts, analyzing the impact of asynchronous communication mechanisms and identifying associated benefits and operational trade-offs.

Rodrigues et al. (2025) introduced a framework to support a systematic literature review of architecture, performance and scalability in microservices. This paper main contribution was the observation that most studies focus on deployment and scaling alternatives for performance and scalability, and are not integrated with performance and scalability assessment.

In summary, to the best of our knowledge, the approach presented in this paper is the first automated framework that integrates static architectural complexity metrics, derived from multiple dependency types, with dynamically measured performance data, enabling the analysis of how microservice complexity impacts performance outcomes.

3. Benchmarks

In this section, we describe the microservice benchmarks we used to evaluate our fully automated PPTAM-DV8 framework. Our objective is to choose non-trivial, open-source microservice systems with at least a dozen service components and multiple publicly available releases to support evolutionary analysis. However, such benchmarks are rare: most open-source systems are either small-scale or lack version history. To establish a baseline, we selected the Hotel Reservation system from DeathStarBench University (2019). While relatively simple and small, it serves as a clean reference point for comparison due to its straightforward architecture and low complexity.

To evaluate the impact of architectural complexity, we selected Train-Ticket L'Aquila (2018), the only non-trivial benchmark available, having over 30 service components and multiple evolutionary releases developed by different teams. Although we planned to analyze more versions, setting up and testing Train-Ticket turned out to be unexpectedly time and resource-intensive in a research environment. Given these constraints, we focused on four representative releases that exhibit clear structural differences. These benchmarks together offer a range of architectural complexity, allowing us to investigate how structural variations influence performance outcomes. Next, we introduce each of these benchmarks in more detail.

3.1. Deathstar hotel reservation

The Hotel Reservation application within DeathStarBench University (2019) provides functionality to search for hotel rooms, retrieve user profiles, get recommendations, and make reservations. The system includes both in-memory and persistent databases and simulates realistic cloud service interactions. It can be deployed using Docker and Docker Compose, with additional support for environments like Kubernetes and OpenShift. It has various configuration flags that allow for tuning of performance and logging behavior.

3.2. SEALAB Qualitygroup Train-Ticket

The SEALAB QualityGroup version of the *Train-Ticket* system L'Aquila (2018) presents an early implementation of a microservices-based railway ticketing platform for research purposes. In this version, each core functionality, such as user management, ticket booking, order processing, and payment, is contained in a dedicated service component, mainly developed in Java using the Spring Boot framework Team (2025). The system employs MongoDB MongoDB (2025) and follows a database-per-service pattern, ensuring data encapsulation and promoting service autonomy.

For interservice communication and external exposure, the system uses Nginx NGINX (2025) as a reverse proxy gateway, which routes external API requests to the appropriate microservices and serves as a unified access point for the platform. Internally, service discovery is managed by Docker Docker (2025), which resolves service locations dynamically without a dedicated service registry. For observability, the system relies on Zipkin OpenZipkin (2025) for trace records to gain insight into run-time behavior and service dependencies. Deployment is supported through Docker Compose for local orchestration, offering flexibility for testing, scaling, and infrastructure management.

3.3. CUHK-SE Group Train-Ticket

The CUHK-SE Group version of the *Train-Ticket* system (Group, 2018) is the latest version of the railway ticketing platform. It has been forked from the same origin as the SEALAB version but using a different database technology: in this variant, services share access to a common MySQL database, following a shared-database pattern that centralizes data management across the system.

Unlike the SEALAB version, which uses Nginx as a reverse proxy gateway, this CUHK-SE version adopts Caddy Holt (2025), a modern general purpose web server, to manage external API traffic and route requests to backend services. Service discovery is handled natively by the Kubernetes infrastructure provided by Minikube Contributors (2025a) which uses DNS-based resolution to dynamically route requests between services without requiring an explicit service registry. Similar to the SEALAB version, this implementation relies on Zipkin for distributed tracing, and integrates OpenTelemetry Contributors (2025c) for collecting and exporting trace data. The deployment leverages Kubernetes through Minikube, using native orchestration features for service management, and Helm charts are used to streamline and automate the deployment process.

3.4. Fudan v0.2.0 Train-Ticket

The Fudan v0.2.0 version of the *Train-Ticket* system (University, 2021) is a later release following the SEALAB version. Its deployment closely resembles that of the SEALAB and CUHK-SE versions, such as modular service decomposition and implementation in Java using the Spring Boot framework. In this variant, each microservice persists its data independently using MongoDB, adhering to the database-per-service pattern. For external exposure, the system employs Nginx as a reverse proxy gateway. Service deployment is managed using Kubernetes Foundation (2025b) in a single-node configuration, offering a more realistic orchestration environment compared to Docker Compose, while remaining lightweight for research scenarios. Observability is supported through Apache SkyWalking Foundation (2025a), which provides distributed tracing, service dependency analysis, and performance monitoring.

3.5. Fudan v1.0.0 Train-Ticket

The Fudan v1.0.0 version of the *Train-Ticket* system (University, 2022) evolved from Fudan v0.2.0 and adopts a Kubernetes-native deployment, executed on a single-node cluster. In contrast to the earlier version v0.2.0, which adopted MongoDB with a database-per-service pattern, this release relies on a shared MySQL cluster, with a scheme per service, deployed via RadonDB in all-in-one mode, providing high availability.

Similar to prior releases, in this version, services are developed in Java using the Spring Boot framework, and the system employs Nginx as the gateway component, exposed through a NodePort service

Table 1
Technology, Deployment, and Database Features Across Five Benchmarks.

Benchmark	Technology							Deployment					Database			
	Go	Java	Spring Boot	gRPC	Nginx	Caddy	Nacos	Helm Charts	Docker	Docker Compose	Kubernetes[a]	Kubernetes[b]	In-memory[c]	MongoDB[d]	MySQL[e]	MySQL[d]
DeathStar Hotel Reservation	×			×					×	×	×		×			
SEALAB *Train-Ticket*		×	×		×				×	×				×		
CUHK *Train-Ticket*		×	×			×		×			×	×			×	
Fudan *Train-Ticket* (v0.2.0)		×	×		×						×			×		
Fudan *Train-Ticket* (v1.0.0)		×	×		×		×	×			×				×	

[a] Deployed on a single node.
[b] Deployed using Minikube.
[c] Persisted to a local database.
[d] Each service with a separate database.
[e] Deployed using a shared database.

to route external requests. Runtime observability is enabled via Apache SkyWalking, which supports distributed tracing and service dependency analysis.

The deployment also integrates NacosGroup (2025), a dynamic service discovery, configuration, and service management platform. Nacos centralizes and manages configuration values (via ConfigMaps and Secrets) and assists microservices in locating each other at runtime, ensuring a consistent and decoupled mechanism for handling dynamic service endpoints and configuration changes. All microservices are defined and orchestrated through declarative Kubernetes manifests, ensuring reproducibility and modularity in deployment.

3.6. Summary

Table 1 summarizes the main technological and deployment characteristics of the five benchmark systems, highlighting their implementation language, the orchestration environment, and the underlying database strategies.

All five systems follow a microservice architecture with modular services handling core functions like user management, booking, and payments. The Train-Ticket systems use Java and Spring Boot, while the DeathStar Hotel Reservation system is written in Go with gRPC. Deployment setups vary: SEALAB uses Docker Compose, while the others adopt Kubernetes via Minikube or single-node clusters, providing a more realistic orchestration environment.

The main difference among these benchmarks lies in their database and gateway strategies. SEALAB and Fudan v0.2.0 use a database-per-service pattern with MongoDB, while CUHK-SE centralizes data in a shared MySQL database. Fudan v1.0.0 also uses MySQL (via RadonDB) but separates schemas per service. DeathStar mixes in-memory and persistent databases based on service needs. Gateway choices also vary: SEALAB and Fudan v0.2.0 use Nginx, CUHK-SE adopts Caddy, and Fudan v1.0.0 continues with Nginx while adding Nacos for configuration and service management.

Most importantly, beyond differences in technology stacks, the key distinction lies in how individual service components are designed to depend on each other through various relationships, such as API calls and shared data structures. As shown in Figs. 7, 8, and 9, these different releases provide similar functionalities, yet their underlying design structures, formed by these dependencies, vary significantly.

Figs. 2, 3, A.10, and A.11 illustrate the call dependencies of each benchmark for visual inspection. Each column corresponds to a microservice, while the arrows indicate its call dependencies on other microservices. The first row shows, for each microservice, the total number of services it calls (fan-out), and the second row shows the total number of services that are called by each microservice (fan-in). Fig. 2 contains an example of a small project, to illustrate the notation of the visualization. Fig. 3 contains the visualization of the SEALAB Trainticket call profile. The visualizations of the call profiles of the remaining studied benchmarking applications are provided in the Appendix.

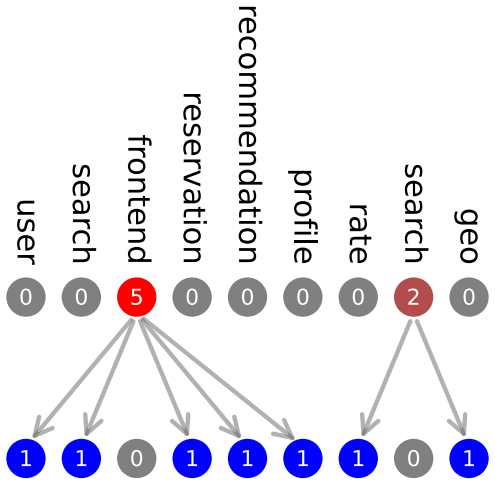


Fig. 2. Call profile of *hotelreservation*.

4. Automated performance and architecture assessment framework

To ensure accurate and unbiased performance evaluation, we established a dedicated testing environment, isolated from other workloads to prevent external interference. We allocated the same amount of computational resources for consistent behavior throughout the test runs. The testbed was designed to be clean and repeatable, enabling reliable comparison of different deployment configurations.

Host-level monitoring was conducted using `vmstat`, allowing us to track system resource metrics such as CPU utilization, memory usage, and I/O activity. This data was critical for correlating host-level resource consumption with specific microservices under load, thereby aiding in the identification of software-induced performance bottlenecks. For each test run of each benchmark, we collected detailed `vmstat` metrics from both the host and virtual machine, along with microservice-level response times. Resource and network pressure analyses are presented in Section 6.2.

This section explains in detail the automation steps depicted in Fig. 1, which consists of three phases: measuring performance outcomes (blue part), measuring architectural complexities (orange part), and analyzing the correlation between the two (green part). Table 2 lists the performance outcomes metrics, while Table 3 lists the architectural metrics.

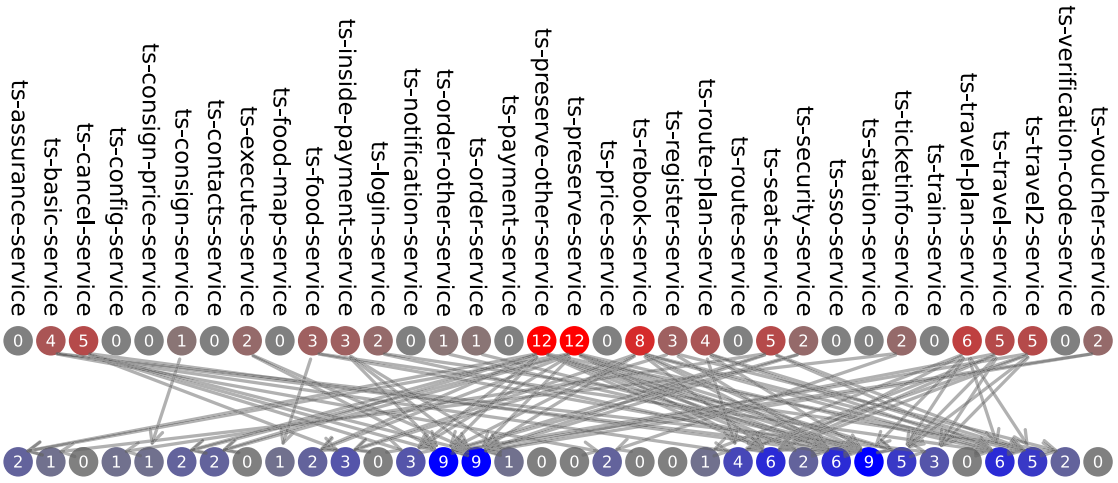


Fig. 3. Call profile of *Trainticket (SEALAB)*.

Table 2
Performance metrics.

Metric Name	Description	Input	Output	Derived
load level	Range of users using the system, controlled by the Test suite PPTAM	×		
$\mathbb{E}[T]$	Average response time in milliseconds		×	
T_max	Maximum response time		×	
$N(F)$	Number of failures detected		×	
slope	The slope is obtained through a linear regression between load levels and corresponding maximum response times.			×
R	Scalability requirement is computed executing a low-load performance test, using the formula $R = \bar{x} + 3\sigma$, where $\bar{x}$ is the mean response time and σ is the standard deviation measured under low load.			×
ND	The normalized distance is derived from the scalability requirement using $ND = 2\frac{T}{T+R}$, where T is the observed response time and R is the scalability requirement.			×

Table 3
Architectural metrics.

Metric Name	Description
Fan out	Fan out is the number of other components or services that a given component directly depends on, indicating how many outgoing dependencies it has and how much coordination it requires
Design Structure Matrix (DSM)	A matrix where rows and columns represent files, packages, or service components, and marked cells indicate dependencies between them.
DSM's Propagation Cost	Measures how many components are likely to be impacted via direct and indirect dependencies.
Clique	Set of components in the DSM that form a strongly connected group, meaning each component directly or indirectly depends on all the others, so a change to one can affect the entire group.
Clique Ratio	Proportion of components in a system that belong to a clique in the Design Structure Matrix, indicating how much of the architecture is tightly and mutually coupled

Fig. 4 illustrates the experimental design: the independent variable is the architecture complexity of the different selected benchmarking systems, as described in Table 3. The dependent variables are the resulting performance and scalability for each system, as described in Table 2. The test condition consists of the selected load levels. The controls are the fixed workload, resources, and tools. The details for each element of the experiment design are illustrated in Table 4.

Measuring performance outcomes. To analyze performance issues, we use PPTAM (Avritzer et al., 2018) to establish the baseline of scalability requirements, calculate the normalized distance of the response time, and identify performance issues:

1. **Baseline scalability requirement**: First, we execute a low load performance tests to derive the scalability requirement response time R_m. For each microservice m, we measure the low load response time $\bar{x}_m$, and its standard deviation σ_m. The scalability requirement R_m is defined as $R_m = \bar{x}_m + 3\sigma_m$. This requirement originates from a well known approach to outlier detection (see, e.g., Barnett and Lewis (1994)), in which values exceeding $\bar{x}_m + 3\sigma_m$ are considered outliers or anomalies, since 99.7% of a Normal distribution lies below this threshold. In our case, we consider performance values beyond that threshold as performance degradation.
2. **Calculate Normalized Distance of the Response Time**: Second, the performance tests are executed with increased load levels to com-

Table 4
Experimental design summary.

Design Element	Description
Research question	What is the relationship between architectural complexity and performance outcomes in microservice systems?
Independent variable	Architecture complexity, defined by each benchmark architecture design.
Test condition	Load level, defined by the number of concurrent users employed in each test.
Dependent variables	Response time (average and maximum), normalized distance (ND), number of failures, and maximum supported users are measured and derived test outputs (see Table 2).
Controls	Defined same workload scenario, run duration, warm-up phase, computer resources, deployment environment for every experiment. Use same analysis tools (PPTAM, DV8).
Experimental groups	Distinct microservice system releases with varying architectural complexity, plus a low-complexity reference system.
Hypothesis	Higher architectural complexity leads to degraded performance and reduced scalability.

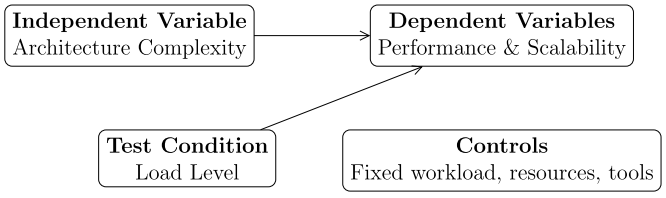


Fig. 4. Experimental design relating microservice architecture complexity to performance and scalability under varying load.

pute the Normalized Distance (ND) of the measured response time to the scalability requirement, $ND = 2 \cdot T/(T + R)$, where T is the measured response time, and R is the scalability baseline scalability requirement computed in Step 1.

3. **Identify microservices with performance issues**: Third, we compute the ND and average response time slope (Avritzer et al., 2018). In Avritzer et al. (2018), the slope of response time, as a function of normal distance (ND), was used to partition the evaluated microservices into four categories: (1) (low risk, low impact) if the slope is small and $ND < 1$, (2) (high risk, low impact) if the slope is large and $ND < 1$, (3) (low risk, high impact) if the slope is small and $ND \geq 1$, (4) (high risk, high impact) if the slope is large and $ND \geq 1$.

We defined a series of performance tests for these benchmarks, using predefined load levels to create distinct test scenarios for evaluating scalability. These load levels were determined by identifying the range within which the software operated without failures. The steps then performed by PPTAM are as follows: (a) Deploy the software under test using Docker or Kubernetes. Depending on the deployment scripts provided with the software, we configured PPTAM to execute the appropriate scripts to deploy and undeploy the system. (b) Execute performance tests and collect response times, resource utilization, and failure rates. PPTAM uses locust (Contributors, 2025b) to run performance tests based on a `locustfile.py`, which defines how the tests should be conducted. PPTAM runs these tests across all configured load levels and collects the runtime data, and (c) Aggregate the collected data at the microservice level. Since our goal is to analyze performance issues per microservice, we defined a configuration mapping between endpoints and microservices for each benchmark system.

The result of this phase includes the calculated normalized distance for each endpoint, the relationship between slope and normalized distance (see Figs. 5 and 6), and detection of performance issues based on the data obtained.

Measuring architectural complexity. Here we leverage DV8 (Cai and Kazman, 2019) to measure architecture complexity. DV8 takes a *Design Structure Matrix* (DSM) (Baldwin and Clark, 1999) as input, visualizes design structures, calculates complexity scores, including Propagation Cost (PC) and Decoupling Level (DL), and identifies design antipatterns. In DSM, rows and columns represent files, packages, or service components, and marked cells indicate dependencies between them. One unique feature of DV8 is that it can integrate multiple types of dependencies. By combining static dependencies with co-change data, DV8 detects design six types of antipatterns, including Unstable Interface (*UIF*), Modularity Violation (*MVG*), and Crossing (*CRS*) (Mo et al., 2021, 2018; Nayebi et al., 2019; Cai et al., 2025a). Without co-change data, it identifies Unhealthy Inheritance (*UIH*), Clique (*CLQ*), and Package Cycle (*PKC*), and reports fan-in/fan-out to highlight overly complex elements.

These DSM-based metrics and antipatterns have been validated in numerous open-source and industrial projects (Mo et al., 2018; Nayebi et al., 2019; Wu et al., 2018; Cai et al., 2025b), demonstrating strong correlations with software maintainability and productivity. In this study, we focus on propagation cost and clique ratio because they have a direct impact on performance: when API calls or data processing involve a large number of components, response time is inevitably degraded. Other DSM-based metrics and antipatterns are either not applicable to microservice systems or not directly related to performance. For example, UIF, MVG, and CRS require co-change data, which are not available, while unhealthy inheritance and package cycles are not applicable in microservice architectures.

Given that this is the first study to examine the relationship between these complexity metrics and performance, exploring empirically grounded thresholds for propagation cost and clique ratio as indicators of poor performance is left to future work. Intuitively, however, when these values exceed 50%, the structure can be considered highly coupled, since changes to any single component would affect more than half of the system's components.

In this study, we first extract API call and entity-sharing relations among various service components, and merge these dependencies into a single DSM, as follows: (a) For each benchmark, we develop static parsers that identify when microservices call each other using http. (b) For each benchmark, identify *imports* relations using jdeps (Oracle, 2025) and identify data dependencies, i.e., situations in which two microservices rely on the same data structure (imported on both sides). (c) The parsing results from both steps are exported in the standard DV8 JSON format and merged into a single DSM by DV8, which is then used to compute complexity scores and detect architectural antipatterns. (d) The identified call dependencies are exported as graphs for visual inspection (see Fig. 2).

Relationship between performance and architecture. We analyzed the relationship between performance outcomes and architectural complexity by comparing normalized distance values, response times, and maximum user load, with overall architecture complexity scores, including propagation cost (MacCormack et al., 2006), clique ratio (Mo et al., 2021), and with individual service complexity measured using endpoint coupling scores (see Tables 10 and 11).

Table 5
Functionalities, API Endpoints for Train-Ticket Benchmark.

Functionality	Service	Relative path from /api/v1/
User login	Users	/users/login
Create contact	Contacts	/contactservice/contacts
Get trip information	Travel	/travelservice/trips/left
Get contacts	Contacts	/contactservice/contacts/account
Get foods	Food	/foodservice/foods
Get insurance types	Assurance	/assuranceservice/assurances/types
Book ticket	Preserve	/preserveservice/preserve
Get last order	Order	/orderservice/order/refresh
Pay ticket	Payment	/inside_pay_service/inside_payment
Collect ticket	Execute	/executeservice/execute/collected
Use ticket in station	Execute	/executeservice/execute/execute

5. Empirical study

For each benchmark, we defined the workload specification as a sequence of requests that emulate realistic user interactions with the system. Each simulated user performs a full end-to-end scenario, covering the main operations a user would execute during a typical scenario, such as ticket booking.

The Train-Ticket scenario includes the following steps: Logging in to obtain authentication tokens and session context; accessing the Homepage to simulate initial navigation; querying for available trips (departure and return); retrieving auxiliary information such as insurance types, food options, and user contacts; selecting a trip and proceeding through the ticket booking process. Once the ticket is successfully booked, the scenario continues with payment for the ticket (if the status is 'booked'), followed by ticket collection and validation for use (executing the travel).

This sequence ensures a complete coverage of the typical ticket booking lifecycle and enables us to evaluate the performance and scalability of the system with realistic user activities. The requests sent during these scenarios are directly mapped to the API endpoints exposed by the system and include the required parameters and session information to simulate authentic behavior.

Table 5 lists all requests involved in these scenarios, together with their respective endpoints and logical groups according to user actions.

Our performance evaluation consisted of testing the endpoints described in Table 5, using the PPTAM tool—an open-source software project (Avritzer et al., 2019a,b). The experimental setup included two primary components: the *Drive* component (PPTAM) operated on a Linux virtual machine with 8 GB RAM and 4 logical CPUs at 2.095 GHz per core and four *Testbed* components. For each testbed, we performed experiments with 1, 2, 4, 6, 8, 10, 15, 20, 25 and 30 concurrent users. The workload scenario corresponding to 1 user serves as the baseline workload to identify the baseline requirement. All tests were executed under the same system conditions, following an initial warm-up phase involving prior test executions. This ensured that services were not in a cold-start state, allowing measurements to reflect steady-state performance characteristics. Each test was run for 20 minutes, resulting in a total testing time of 200 minutes.

The replication package (Avritzer et al., 2025a) contains the source code used to generate the results in this paper, detailed measurement data, structural data, calls structure, data coupling, and additional graphs that are not shown in the paper: call graphs, normalized distance vs. load, and vmstat graphs. This testing campaign was a significant effort carried over several months by experts in performance, scalability and architecture analysis.

6. Performance and architecture analyses

In this section, we present the performance and architecture analysis outcomes from our empirical study.

6.1. Virtual machines response times

Here we present the most representative performance results from SEALAB (Table 6), Fudan v0.2.0 (Table 7), and CUHK-SE (Table 8). The slope versus normalized distance results for all Train-Ticket systems and the Hotel Reservation system are shown in Fig. 5 and Fig. 6, respectively. Table 9 summarizes the endpoints, normalized distances, and corresponding microservices for Fudan v0.2.0, Fudan v1.0.0, and CUHK-SE. Since the endpoints of these releases share the same names, we begin by elaborating on their results, followed by those of SEALAB and the Hotel Reservation system.

Fudan v0.2.0 Train-Ticket. Performance results for this version are shown in Table 7 and Fig. 5(a). Table 7 contains performance results for 1, 10, and 15 users. The performance results for the 15 users load level show a significant number of failures for the *preserve-service* (10) and very large response times, which indicates that the system was unable to scale beyond fifteen users. Fig. 5(a)(a) highlights significant degradation in the *preserve-service* (10), as indicated by its normalized distance and response time slope.

CUHK-SE Train-Ticket. Performance data for this version are shown in Table 8 and Fig. 5(c). Table 8 contains performance results for 1, 10, and 20 users. Notable degradation is observed in *food-service* (7), *payment-service* (8), *order-service* (9), *preserve-service* (10), and *travel-service* (11), as reflected in their high normalized distances and steep response time slopes (Fig. 5(c)), and very large expected and max response times for the 20 users load level (Table 8).

Fudan v1.0.0 Train-Ticket. The performance results of this version (not shown) is similar to that of CUHK-SE (Table 8), and could support up to twenty users. The *preserve-service* (10) again shows significant performance issues, as illustrated by Fig. 5(b) with multiple microservices reaching normalized distance values near the 2.0 boundary at this load level.

SEALAB Train-Ticket. Results for this version are presented in Table 6 and Fig. 5(d). This system displayed severe performance degradation at very low load level, failing to scale beyond just two users. The *preserve-service* (12) and *travel-service* (13) showed especially poor behavior, with high normalized distances and steep response time slopes even at the minimal load level of 2 users (Fig. 5(d)(d)), and very large expected and max response times (Table 6).

DeathStar Hotel Reservation. The test results of this system are shown in Fig. 6. They show that this benchmark is scalable for the range of load levels measured, because the normalized distance is less than one for all service components and the slope remains small in the range of -0.2 to 1.2.

In summary, while the high scalability of the Hotel Reservation system can be attributed to its simplicity, the four releases of the Train-Ticket microservice system with similar functions exhibited drastically different performance outcomes.

6.2. Physical host performance

We recorded average CPU utilizations, memory occupancy, I/O activity, context switching rates, and interrupt rates during successive ten second intervals using *vmstat* for both the physical host and within the single virtual machine on which *Train-Ticket* was run. The enforced absence of other activity on the physical host ensured clean measurements

Table 6
SEALAB *Train-Ticket* performance results. xx indicates a failed test.

Path	Test 1 (1 user) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 2 (2 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 4 (6 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$
/login	66.84	66.84	0	72.99	77.65	0	99.42	112.96	0
/contacts/create	67.22	67.22	0	22.10	27.04	0	30.42	36.98	0
/travel/query	366.46	559.16	0	307.00	398.63	0	1182.19	1952.75	0
/contacts/findContacts	20.52	35.52	0	18.21	33.42	0	15.40	25.32	0
/assurance/getAllAssuranceType	9.52	22.68	0	8.33	20.13	0	9.59	20.32	0
/food/getFood	60.86	101.78	0	51.35	68.20	0	55.80	89.77	0
/preserve	592.40	1213.18	0	31559.65	60065.49	36	60035.80	60083.25	114
/order/queryForRefresh	136.32	312.50	0	25.68	40.55	0	xx	xx	xx
/inside_payment/pay/	50.94	401.53	0	39.26	48.63	0	xx	xx	xx
/execute/collected	24.82	40.36	0	19.87	31.56	0	xx	xx	xx
/execute/execute	24.17	41.61	0	17.52	25.41	0	xx	xx	xx
Aggregated	141.70	1213.18	0	5194.46	60065.49	36	36884.10	60083.25	114

Table 7
Fudan v0.2.0 *Train-Ticket* performance results.

Relative path from /api/v1	Test 1 (1 user) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 6 (10 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 7 (15 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$
/users/login	2823.64	5495.79	0	2680.06	3885.74	0	3797.03	9703.70	0
/contactservice/contacts/ account/user	16.89	31.92	0	8.85	47.85	0	8.96	14.58	0
/travelservice/trips/left	966.55	60068.17	1	2613.38	16605.83	0	7272.07	60012.93	2
/contactservice/contacts	4486.83	4486.83	0	12.41	19.11	0	12.88	22.61	0
/assuranceservice/assurances/types	39.62	4141.64	0	8.26	71.46	0	8.56	36.22	0
/foodservice/foods/	207.13	13989.11	0	37.93	395.83	0	40.66	127.44	0
/preserveservice/preserve	839.53	17147.60	0	21758.64	47667.52	0	58774.59	60031.60	266
/orderservice/order/refresh	121.34	717.76	0	786.90	5799.69	0	2188.82	13301.61	0
/inside_pay_service/inside_payment	159.63	12814.37	0	803.99	7304.95	0	1913.49	6103.88	0
/executeservice/execute/collected/order	105.72	11613.38	0	552.56	3402.89	0	921.92	6902.55	0
/executeservice/execute/order	46.40	466.54	0	542.50	12108.89	0	1542.63	7106.19	0
Aggregated	297.36	60068.17	1	2394.90	47667.52	0	26333.10	60031.60	268

Table 8
CUHK-SE *Train-Ticket* performance results.

Relative path from /api/v1	Test 1 (1 user) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 6 (10 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$	Test 8 (20 users) $\mathbb{E}[T]$ (ms)	T_max (ms)	$N(F)$
/users/login	126.69	161.58	0	163.94	208.87	0	223.96	344.74	0
/contactservice/contacts/account/user	17.55	36.00	0	10.83	29.35	0	13.35	61.30	0
/travelservice/trips/left	183.28	431.60	0	2478.16	22907.77	1	15318.02	53053.67	10
/contactservice/contacts	148.94	148.94	0	27.25	51.27	0	21.41	67.09	0
/assuranceservice/assurances/types	22.67	2420.70	0	9.25	32.02	0	11.20	45.38	0
/foodservice/foods/	68.75	151.63	0	34.07	105.68	0	2604.84	28128.80	0
/preserveservice/preserve	332.27	539.17	0	5063.50	23707.25	1	34769.36	57195.09	4
/orderservice/order/refresh	17.16	82.63	0	95.50	6238.24	0	686.70	12232.73	0
/inside_pay_service/inside_payment	66.23	153.49	0	232.70	4567.65	0	1799.67	24958.50	0
/executeservice/execute/collected/order	31.45	73.50	0	140.45	8634.27	0	538.35	11579.05	0
/executeservice/execute/execute/order	31.69	181.46	0	102.84	2122.53	0	342.22	14535.79	0
Aggregated	77.62	3172.71	0	832.09	23707.25	2	5530.95	57195.09	14

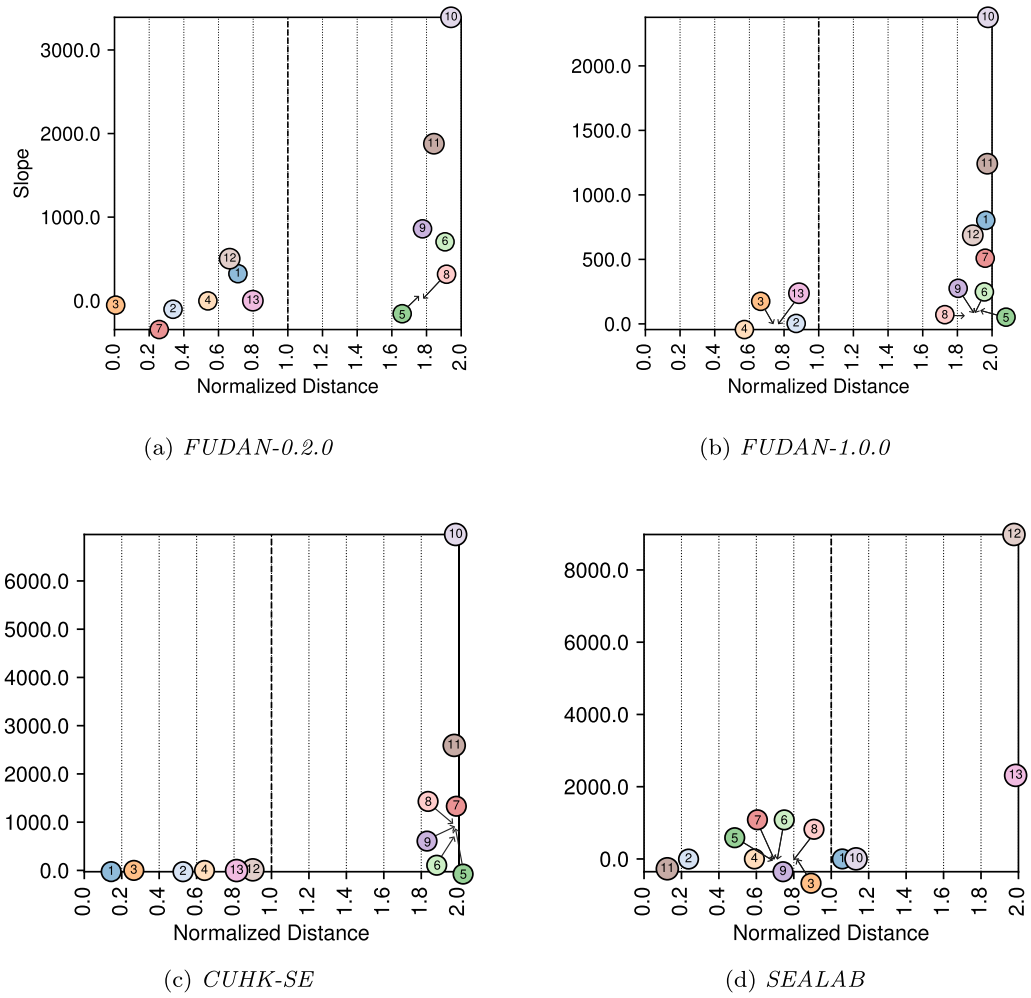


Fig. 5. Slope vs Normalized Distance for four releases of TrainTicket systems. The numbers in the figures correspond to endpoints.

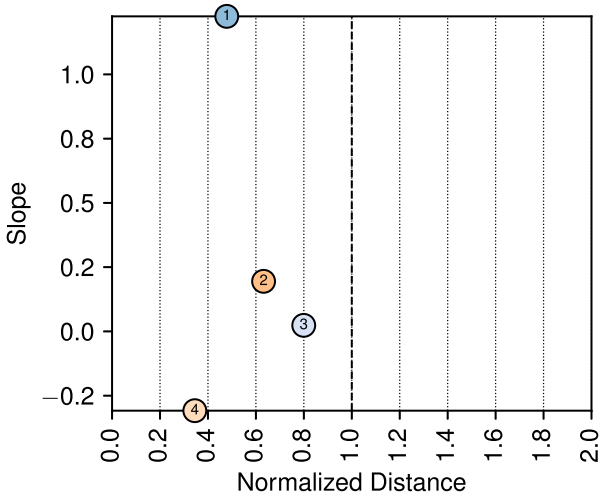


Fig. 6. Slope vs Normalized Distance *hotelreservation*. The numbers in the figure correspond to endpoints.

of the resources used by the virtual machine. We developed a Python tool to automatically plot the contents of all of our *vmstat* output logs using *ggplot*.

Our measurements show that the physical machine did not constrain the execution of the virtual machine. There was very little resource contention in the physical machine. All resource contention was contained within the virtual machine. The user CPU utilization on the physical machine never exceeded 12.5%. The system CPU on the physical machine was not visible in the plots. Memory occupancy and the number of idle pages on the physical machine were constant and never a cause for concern, even though they varied on the virtual machine. The interrupt and context switching rates on the physical machine were always much lower than the corresponding rates within the virtual machine. Similarly, measured I/O activity measured in blocks in and blocks out per second was always much smaller in the physical machine than in the virtual machine. No process swapping was observed in either the physical machine or the virtual machine.

The results indicate that overall performance is not constrained by the physical host, suggesting that the primary factor influencing performance is the architectural complexity of the benchmark systems.

6.3. Architecture analysis results

To analyze the overall structure of the service components in each microservice architecture, we extracted two dependency types—API calls and data dependencies—and constructed design structure matrices (DSMs) for all four releases of the TrainTicket system. Figs. 7 and 8 show the DSMs for SEALab and Fudan 0.2.0, respectively. Because the DSMs for Fudan 1.0.0 and the CUHK versions are identical, we only present the CUHK matrix in Fig. 9. Using DV8, we also identified an-

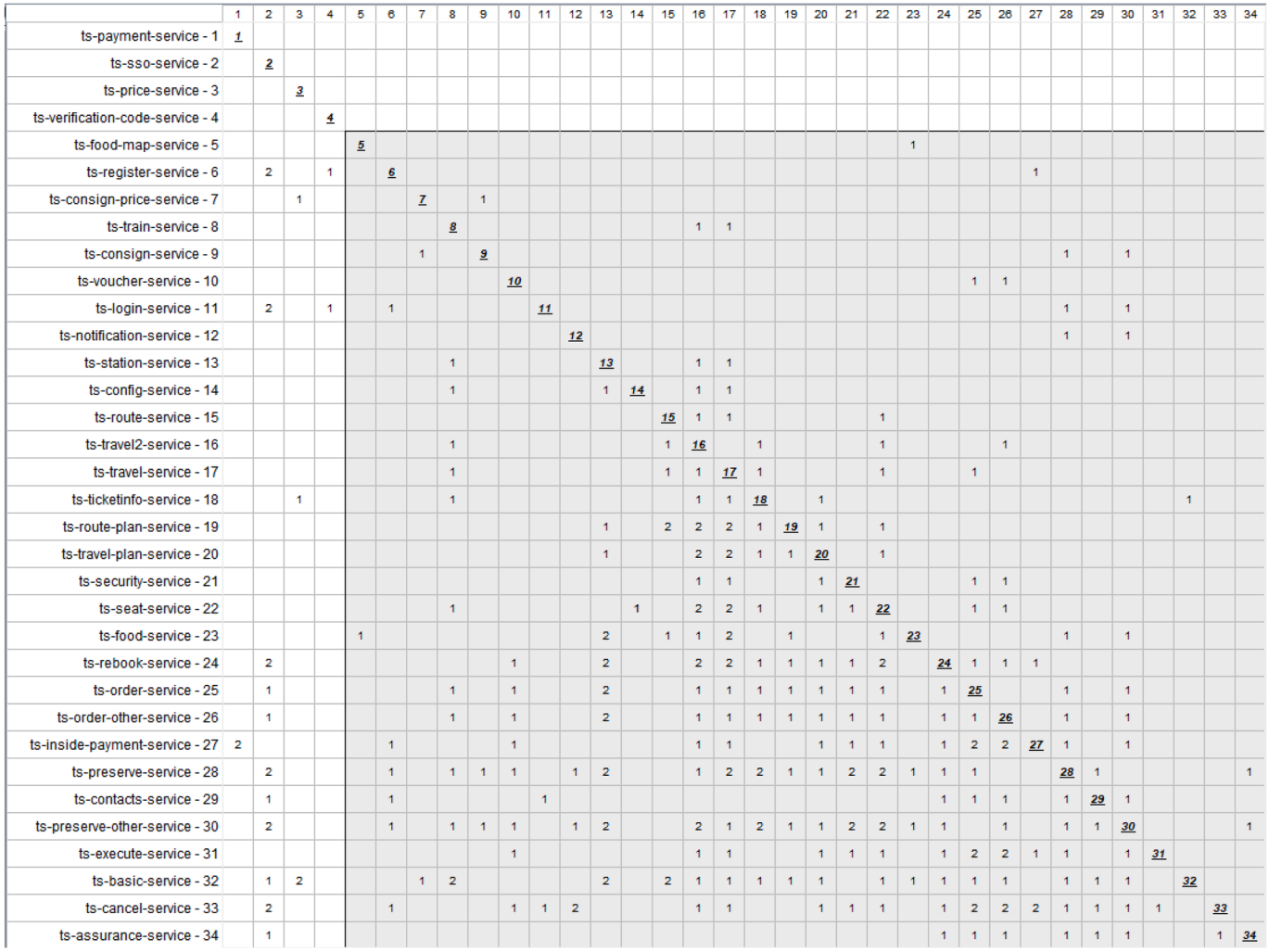


Fig. 7. The Evolution of TrainTicket: SEALab DSM and Clique.

Table 9
Normalized Distance (ND) Values for Endpoints Across Three TrainTicket Systems.

#	Endpoint	Norm. Distance CUHK	FUDAN 0.2.0	FUDAN 1.0	Microservice
1	adminuserservice/users	0.144	0.712	1.964	None
2	assuranceservice/types	0.527	0.338	0.870	ts-assurance-service
3	contactservice/contacts	0.265	0.007	0.737	ts-contacts-service
4	contacts/account/user	0.642	0.539	0.571	ts-contacts-service
5	executeservice/collected/order	1.980	1.753	1.929	ts-execute-service
6	executeservice/execute/order	1.973	1.907	1.901	ts-execute-service
7	foodservice/foods	1.987	0.258	1.960	ts-food-service
8	inside_pay_service/payment	1.965	1.765	1.837	ts-inside-payment-service
9	orderservice/refresh	1.977	1.777	1.889	ts-order-service
10	preserveservice/preserve	1.984	1.941	1.976	ts-preserve-service
11	travelservice/trips/left	1.976	1.843	1.972	ts-travel-service
12	users/login	0.900	0.664	1.889	ts-user-service
13	index.HTML	0.813	0.798	0.772	None

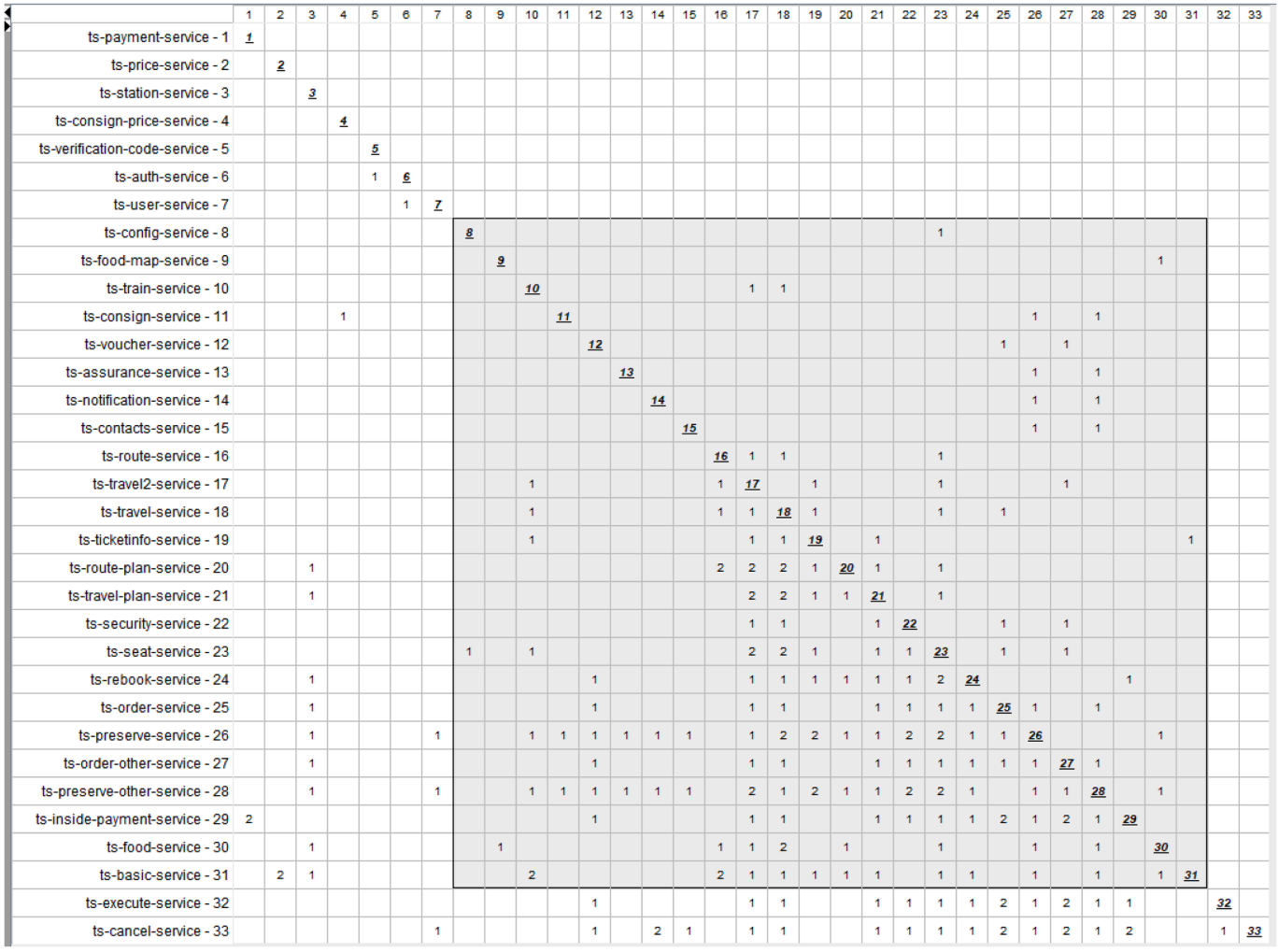


Fig. 8. The Evolution of TrainTicket: Fudan 0.2.0 DSM and Clique.

Table 10
Design Structure and Performance/Scalability.

Benchmark	#Services	MaxUsers	Response Time		Propagation Cost	#CliqueFiles	Clique Ratio
			Avg	Max			
SEALab	34	2	5194.46	60065.49	88.60%	30	88.20%
Fudan 0.2.0	33	15	153.94	3607.48	75.20%	24	72.70%
Fudan 1.0.0	35	20	67.82	1575.58	47.80%	15	42.90%
CUHK-SE	35	20	64.66	807.90	47.80%	15	42.90%
hotelreservation	4	30	11.27	26.74	21.88%	0	0%

tipatterns within each DSM. Each version contains exactly one Clique, comprising 30 services in SEALAB, 24 services in Fudan v0.2.0, and 15 services in both Fudan v1.0.0 and CUHK-SE. The Clique anti-pattern indicates that dependencies among the components form a strongly connected graph, such that a change to any one component may potentially affect all the others. In the DSMs, the gray blocks highlight the services that participate in these cliques. As Fig. 7 shows, SEALab's clique involves the largest proportion of its components, while the Fudan 1.0.0/CUHK-SE release has the fewest services captured in the clique.

To quantify the degree of coupling among these services, we computed each DSM's propagation cost (PC) (MacCormack et al., 2006; Mo et al., 2016). The PC measures, for any given component change, how many other components are likely to be impacted via direct and indirect dependencies. Concretely, PC is the ratio of the total number of dependencies in the DSM's maximum transitive closure (that is, the count of all direct and indirect dependencies) to the total possible number of dependencies (i.e., the square of the matrix size).

Table 10 summarizes, for each benchmark, the number of service components in the clique, the clique ratio, and the propagation cost (PC) score. This table also presents the maximum number of users the system can support given the same amount of available resources, as well as other performance-related metrics, which we will elaborate on in the next Section. As a reference, the Hotel Reservation system has

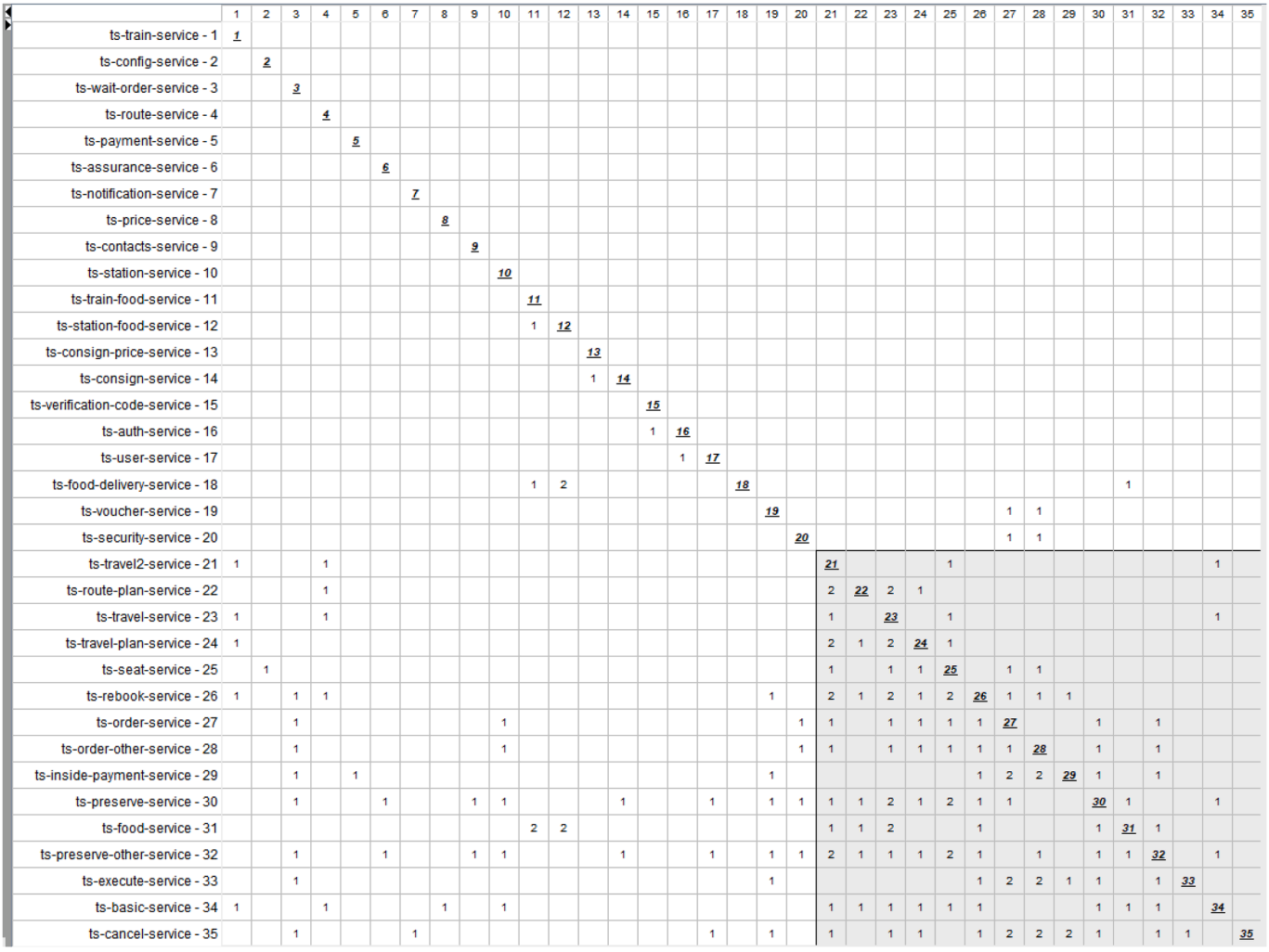


Fig. 9. The Evolution of TrainTicket: Fudan 1.0.0 and CUHK-SE, DSM and Clique.

no cliques and a PC score of only 22%. Its simplicity contributes to the shortest response time and makes it the only system that handled all 30 users, the maximum tested across all systems, without encountering any exceptions.

Although the strong performance and scalability of the Hotel Reservation system can be attributed to its functional simplicity, the four versions of TrainTicket, despite having similar functionalities, exhibited drastically different architectural structures and levels of complexity. Table 10 shows that although the four releases of TrainTicket have a similar number of services (ranging from 33 to 35), their architectural structures differ significantly, as evidenced by their PC scores and the proportion of components involved in a clique (*Clique Ratio*). SEALab's PC score is nearly double those of Fudan 1.0.0 and CUHK-SE, indicating a much higher degree of architectural complexity. In SEALab, the majority of service components (88.20%) are involved in a Clique antipattern, suggesting that despite adopting a microservice architecture, few components are truly independently evolvable.

In addition to Propagation Cost (PC) and cliques, which we used to measure the complexity of the overall microservice architecture, we also analyzed the coupling level of individual endpoints. Intuitively, the fan-out of an endpoint, defined as the number of other endpoints it depends on, has a significant impact on its performance outcomes, such as normalized distance. The last column of Table 11 lists the fan-out scores for each endpoint.

7. Correlating complexity and performance

Since this automated framework enabled us to quantify performance outcomes, such as normalized distances (ND) and response time, and generate a DSM representing the overall architectural structure, it became possible to analyze the correlation between the architectural complexity of a microservice system, formed by multiple static relations among service components, and its runtime performance. In this study, we assess architectural complexity from two perspectives: the coupling (fan-out) score of each individual endpoints, and the overall architectural complexity measured by propagation costs and clique ratio calculated using a combination of data dependency and API call dependency.

Table 11 presents the ND and Coupling scores of all 48 service endpoints we studied from the five benchmark systems. To determine the appropriate statistical test, we performed a Shapiro-Wilk test and confirmed that the data are not normally distributed. Accordingly, we applied Spearman's correlation analysis to evaluate the relationship. The results yielded a correlation coefficient of 0.6781 with a p-value of 1.19×10^{-7}, indicating a statistically significant positive correlation between the performance scores of individual services and their coupling with other services. The results align with our intuition: the more tightly coupled a service component is, the poorer its performance.

We aim to evaluate the overall performance across all service components using metrics such as average and maximum response times

Table 11
Normalized Distance and Coupling Correlation.

Microservice	ND	Fan-Out[a]
CUHK		
ts-assurance-service:types	0.527	0
ts-contacts-service:contacts	0.265	0
ts-contacts-service:user	0.642	0
ts-execute-service:collected/order	1.980	2
ts-execute-service:execute/order	1.973	2
ts-food-service:D1345	1.987	3
ts-inside-payment-service:inside_payment	1.965	3
ts-order-service:refresh	1.977	1
ts-preserve-service:preserve	1.984	11
ts-travel-service:left	1.976	4
ts-user-service:login	0.900	1
FUDAN 0.2.0		
ts-assurance-service:types	0.338	0
ts-contacts-service:contacts	0.007	0
ts-contacts-service:user	0.539	0
ts-execute-service:collected/order	1.753	2
ts-execute-service:execute/order	1.907	2
ts-food-service:D1345	0.258	3
ts-inside-payment-service:inside_payment	1.765	2
ts-order-service:refresh	1.777	0
ts-preserve-service:preserve	1.941	12
ts-travel-service:left	1.843	5
ts-user-service:login	0.664	1
FUDAN 1.0.0		
ts-assurance-service:types	0.870	0
ts-contacts-service:contacts	0.737	0
ts-contacts-service:user	0.571	0
ts-execute-service:collected/order	1.929	2
ts-execute-service:execute/order	1.901	2
ts-food-service:D1345	1.960	3
ts-inside-payment-service:inside_payment	1.837	3
ts-order-service:refresh	1.889	1
ts-preserve-service:preserve	1.976	11
ts-travel-service:left	1.972	4
ts-user-service:login	1.889	1
SEALAB		
ts-assurance-service:getAllAssuranceType	0.815	0
ts-contacts-service:create	0.590	0
ts-contacts-service:findContacts	0.687	0
ts-execute-service:collected	0.712	2
ts-execute-service:execute	0.693	2
ts-food-service:getFood	0.804	3
ts-inside-payment-service:pay	0.742	3
ts-login-service:login	1.132	2
ts-order-service:queryForRefresh	0.124	1
ts-preserve-service:preserve	1.976	12
ts-travel-service:query	1.985	5
Hotel		
Search:hotels	0.478	1
Recommendation	0.632	1
User:login	0.800	0
Reservation:reservation	0.345	0
Correlation coefficient: **0.6781** p-value: **1.19E-07**		

[a] Coupling is measured by fan-out, assuming that services with more dependencies face greater performance degradation from coordination overhead.

with two users (the maximum load for SEALAB) aggregated from PPTAM data (last row of Tables 6, 7, and 8), and the maximum number of users supported without exceptions, summarized in Table 10. While we prefer to test more releases to achieve statistically significant results, only four `Train-Ticket` releases were deployable and testable within our resource constraints.

Table 10, though based on only four data points, shows that higher architectural complexity, measured by PC and Clique Ratio, severely degrades performance and scalability. Systems with higher PC values experience dramatically worse performance; for instance, SEALAB (PC = 88.6%) shows average and maximum response times dozens of times higher than those of less-coupled systems like CUHK-SE (PC = 47.8%). Additionally, the maximum number of supported users varies significantly: SEALAB encounters exceptions once the number of users exceeds two, whereas Fudan 1.0.0 and CUHK-SE can support up to 20 users before experiencing similar issues, highlighting the profound scalability penalty associated with increased architectural coupling. Testing more systems and more releases is our future work.

8. Threats to validity

In this work, we have implemented a new fully automated approach to assess the relationship between the performance outcomes and architecture complexities of microservice systems.

In this section, we discuss four potential threats of validity based on a standard checklist developed by Wohlin (2014). We discuss internal, construct, external, and conclusion validity.

Internal validity. We only evaluated four different variants of the *Train-Ticket* benchmark, under a fixed small number of users, and consistently identified correlations between performance outcomes and architecture complexities for all of them. These results are encouraging, although further validation is necessary in large-scale industrial systems with more releases to generalize this result. However, using the four different variants of the *Train-Ticket* benchmark allowed us to control for factors that could have influenced the experimental results.

Construct validity. Our analysis relies exclusively on *Train-ticket* measurements, on DV8 for architecture complexity assessment, and on PPTAM for performance testing. While these tools are well-suited for their respective purposes, it is unclear whether the results would hold if alternative architecture analysis or performance testing tools were used. However, *Train-ticket* has been extensively used in the microservice research community to evaluate architecture and performance. A recent literature review based analysis of 378 Microservices-based Open-Source Projects shows that the number of microservices in such projects ranges from 3 to 50 (d'Aragona et al., 2024). Train-ticket has 42 microservices and was ranked third in this analysis.

External validity. The benchmarks analyzed in this study rely on specific technology stacks, such as Java with Spring Boot or Go with gRPC. As a result, it is unclear whether our findings will generalize to microservice architectures built with significantly different technologies, frameworks, or deployment environments. Further evaluation on diverse technology stacks is needed to strengthen external validity.

Conclusion validity. There is a threat to validity concerning the use of only two tools: PPTAM and DV8. These are high level tools that were used to assess performance and architecture complexity metrics. We plan to extend our work using additional performance, scalability, and architecture metrics. Differences in metric definitions, detection algorithms, or measurement granularity may affect the observed correlations.

9. Conclusion

We have presented a fully automated framework that assesses the relationship between microservice architecture complexity and performance-related quality attributes. Our framework integrates two tools: PPTAM, a performance testing tool used to evaluate system response time under varying user loads; and DV8, an architecture analysis tool used to compute structural complexity metrics, including coupling scores, propagation cost, and architectural antipatterns derived from multiple types of static dependencies.

We evaluated this framework on five benchmark systems, including four structurally distinct versions of the Train-Ticket microservice system, which were developed by different teams using different architecture choices for database and deployment that implemented similar functionalities. The performance testing revealed significant disparities in scalability. For instance, the *SEALAB* Train-Ticket version exhibited severe degradation early on and failed to scale beyond two users, while the *Fudan v0.2.0*, *Fudan v1.0.0*, and *CUHK-SE* versions scaled to fifteen or twenty users with varying levels of performance. To analyze microservice architecture complexity, we first statically extracted API call dependency and data dependencies among service components, and then merge these dependencies into a design structure matrix (DSM) model that can be analyzed using DV8, which calculates propagation costs and detects antipatterns from each combined dependency model.

The analysis revealed that in all four Train-Ticket architectures evaluated, the endpoint with the highest coupling score was always the one with the longest response time. We applied the Spearman's correlation analysis to evaluate the relationship between the coupling score of individual service endpoints and their performance, and we have found a statistically significant positive correlation between the two. The results also show that systems with poor values of complexity metrics, specifically, high propagation cost (PC) and high Clique ratios, demonstrate limited scalability. Notably, the PC score of *SEALAB*, which showed the worst scalability, was nearly double that of *Fudan v1.0.0* and *CUHK-SE*.

This work presents the first fully automated approach that connects statically extracted architectural complexity, captured through heterogeneous dependency types, with dynamically measured performance outcomes in microservice systems and it opens the door to several promising research directions.

Our current framework extracts only two types of dependencies: data and API calls. However, other forms of dependencies, such as configuration and build dependencies, may also influence important architectural quality attributes beyond performance, such as resource consumption and maintainability.

Ultimately, architectural design is a trade-off analysis. Designers must balance modularity, performance, scalability, availability, and more. We plan to significantly extend this framework to quantify a broader range of quality attributes and support data-driven trade-off analysis among them.

We are planning to collaborate with industry partners to evaluate the framework on large-scale real-world systems. Our goal is to explore whether this framework can also monitor architectural evolution over time and detect emerging architecture anti-patterns that may threaten system performance and scalability.

In summary, we are investigating several extensions to our fully automated approach: enhancing PPTAM to automatically detect performance antipatterns, validating our tools on large-scale industrial systems, and enabling DV8 to dynamically construct dependency structure matrices (DSMs) from runtime call graphs.

CRediT authorship contribution statement

Alberto Avritzer: Writing – review & editing, Writing – original draft, Methodology, Conceptualization; **Andrea Janes:** Writing – review & editing, Writing – original draft, Visualization, Methodology, Conceptualization; **Helena Rodrigues:** Writing – review & editing, Writing – original draft, Investigation, Data curation; **Yuanfang Cai:** Writing – review & editing, Writing – original draft, Investigation, Data curation; **Teiji Schoyen:** Data curation; **Ernst Pisch:** Writing – review & editing, Investigation, Data curation; **Catia Trubiani:** Writing – review & editing, Investigation, Data curation; **Andre B. Bondi:** Writing – review & editing, Investigation, Data curation; **Daniel Sadoc Menasché:** Writing – review & editing, Investigation, Data curation; **Chenxi Zhang:** Data curation.

Data availability

We provide all the data in tables, charts, and figures as well as release dependency files in a replication package at Avritzer et al. (2025a). The used research data is shared at https://doi.org/10.5281/zenodo.15528240.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We would like to thank the anonymous reviewers for their valuable feedback. C. Trubiani is partially funded by the MUR-PRIN project 20228FT78M DREAM, MUR Department of Excellence 2023 - 2027 for GSSI, and PNRR ECS00000041 VITALITY.

Appendix A. Train-ticket Call Profiles

In this section we illustrate in Fig. A.10 the call profile for the FUDAN version and in Fig. A.11 the call profile for CUHK-SE version of Train-ticket.

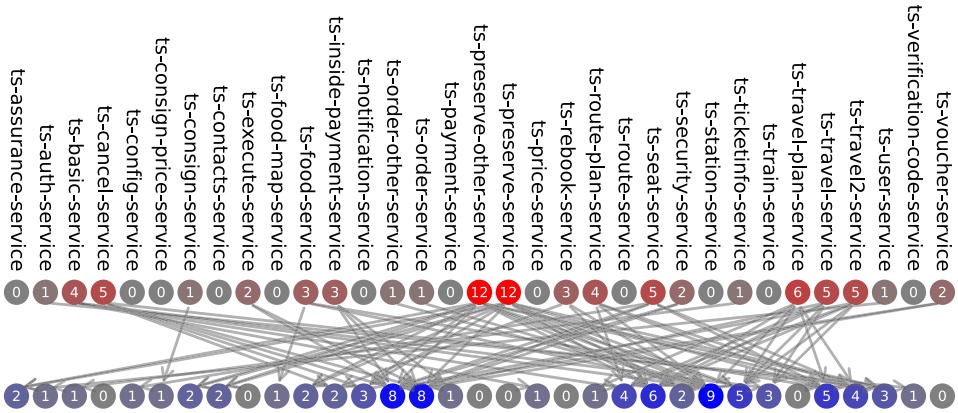


Fig. A.10. Call profile of *Trainticket (FUDAN)*.

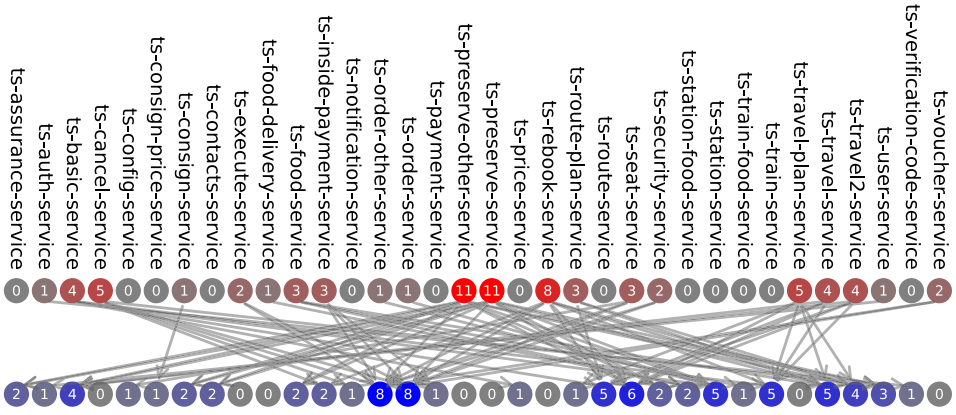


Fig. A.11. Call profile of *Trainticket (CUHK-SE)*.

References

Avritzer, A., Britto, R., Trubiani, C., Camilli, M., Janes, A., Russo, B., van Hoorn, A., Heinrich, R., Rapp, M., Henß, J., Chalawadi, R.K., 2022. Scalability testing automation using multivariate characterization and detection of software performance antipatterns. J. Syst. Softw. 193, 111446. https://doi.org/10.1016/j.jss.2022.111446.

Avritzer, A., Britto, R., Trubiani, C., Russo, B., Janes, A., Camilli, M., van Hoorn, A., Heinrich, R., Rapp, M., Henß, J., 2021. A multivariate characterization and detection of software performance antipatterns. In: Proceedings of the International Conference on Performance Engineering (ICPE), p. 61–72.

Avritzer, A., Ferme, V., Janes, A., Russo, B., Schulz, H., van Hoorn, A., 2018. A quantitative approach for the assessment of microservice architecture deployment alternatives by automated performance testing. Springer. Software Architecture: 12th European Conference on Software Architecture, ECSA 2018, Madrid, Spain, September 24–28, 2018, Proceedings 12, 159–174.

Avritzer, A., Janes, A., Rodrigues, H., Cai, Y., Schoyen, T., Pisch, E., Trubiani, C., Bondi, A.B., Menasché, D., Zhang, C., 2025a. Replication Package and Appendix for the Paper "Automated Assessment of the Relationship Between Microservice Architectures and Performance". Accessed 27 May 2025. https://doi.org/10.5281/zenodo.15528241.

Avritzer, A., Janes, A., Trubiani, C., Rodrigues, H., Cai, Y., Menasché, D.S., de Oliveira, Á. J.A., 2025b. Performance modeling and analysis of design patterns for microservice systems. In: 2025 IEEE 20th International Conference on Software Architecture (ICSA), pp. 35–46. https://doi.org/10.0000/ICSA.2025.

Avritzer, A., Menasché, D., Rufino, V., Russo, B., Janes, A., Ferme, V., van Hoorn, A., Schulz, H., 2019a. PPTAM: Production and performance testing based application monitoring. In: Companion of the 2019 ACM/SPEC International Conference on Performance Engineering. Association for Computing Machinery, New York, NY, USA, p. 39–40. https://doi.org/10.1145/3302541.3311961.

Avritzer, A., Menasché, D., Rufino, V., Russo, B., Janes, A., Ferme, V., van Hoorn, A., Schulz, H., 2019b. Production and Performance Testing based Application Monitoring (PPTAM). Accessed 27 May 2025. https://github.com/pptam/pptam-tool.

Bakhtin, A., Esposito, M., Lenarduzzi, V., Taibi, D., 2025. Network centrality as a new perspective on microservice architecture. Proceedings of International Conference on Software Architecture (ICSA).

Baldwin, C.Y., Clark, K.B., 1999. Design Rules: The Power of Modularity Volume 1. MIT Press, Cambridge, MA, USA.

Barnett, V., Lewis, T., 1994. Outliers in Statistical Data. Wiley, New York. 3rd edition.

Berry, V., Castelltort, A., Lange, B., Teriihoania, J., Tibermacine, C., Trubiani, C., 2024. Is it worth migrating a monolith to microservices? an experience report on performance, availability and energy usage. In: Proceedings of the International Conference on Web Services (ICWS), pp. 944–954.

Blinowski, G., Ojdowska, A., Przybyłek, A., 2022. Monolithic vs. microservice architecture: a performance and scalability evaluation. IEEE Access 10, 20357–20374.

Bosilia, N., Weinberger, G., Haindl, P., 2025. Assessing the impact of asynchronous communication on resilience and robustness: a comparative study of microservice and monolithic architectures. In: Proceedings of the European Conference on Software Architecture (ECSA), pp. 171–186.

Cai, Y., He, L., Qian, J., Kochinski, Y., Zhang, N., Jaspan, C., Bianco, A., 2025a. Understanding architectural complexity, maintenance burden, and developer sentiment–a large-scale study. In: Proceedings of the International Conference on Software Engineering (ICSE), pp. 705.

Cai, Y., He, L., Qian, J., Kochinski, Y., Zhang, N., Jaspan, C., Bianco, A., 2025b. Understanding architectural complexity, maintenance burden, and developer sentiment—a large-scale study. In: Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. IEEE Press, p. 2176–2187. https://doi.org/10.1109/ICSE55347.2025.00168.

Cai, Y., Kazman, R., 2019. DV8: Automated architecture analysis tool suites. In: 2019 IEEE/ACM International Conference on Technical Debt (TechDebt), pp. 53–54. https://doi.org/10.1109/TechDebt.2019.00015.

Camilli, M., Guerriero, A., Janes, A., Russo, B., Russo, S., 2022a. Microservices integrated performance and reliability testing. In: Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test. Association for Computing Machinery, New York, NY, USA, p. 29–39. AST '22, https://doi.org/10.1145/3524481.3527233.

Camilli, M., Janes, A., Russo, B., 2022b. Automated test-based learning and verification of performance models for microservices systems. J. Syst. Softw. 187, 111225. https://doi.org/10.1016/j.jss.2022.111225.

Contributors, K., 2025a. Minikube: Run Kubernetes Clusters Locally. Accessed 27 May 2025. https://minikube.sigs.k8s.io/.

Contributors, L., 2025b. Locust: A Modern Load Testing Framework. Accessed 27 May 2025. https://locust.io.

Contributors, O., 2025c. Opentelemetry: Observability for Cloud-Native Software. Accessed 27 May 2025. https://opentelemetry.io/.

d'Aragona, D.A., Bakhtin, A., Li, X., Su, R., Adams, L., Aponte, E., Boyle, F., Boyle, P., Koerner, R., Lee, J., Tian, F., Wang, Y., Nyyssölä, J., Quevedo, E., Rahaman, S.M., Abdelfattah, A.S., Mäntylä, M., Cerny, T., Taibi, D., 2024. A dataset of microservices-based open-source projects. In: Proceedings of the 21st International Conference on Mining Software Repositories. Association for Computing Machinery, New York, NY, USA, p. 504–509. MSR '24, https://doi.org/10.1145/3643991.3644890.

De Camargo, A., Salvadori, I., Mello, R.d.S., Siqueira, F., 2016. An architecture to automate performance tests on microservices. In: Proceedings of the 18th International Conference on Information Integration and Web-based Applications and Services, pp. 422–429.

Detti, A., Funari, L., Petrucci, L., 2023. μBench: an open-source factory of benchmark microservice applications. IEEE Trans. Parallel Distrib. Syst. 34, 968–980.

Docker, I., 2025. Docker: Accelerated Container Application Development. Accessed 27 May 2025. https://www.docker.com/.

Engel, T., Langermeier, M., Bauer, B., Hofmann, A., 2018. Evaluation of microservice architectures: a metric and tool-based approach. In: Proceedings of the International Conference on Advanced Information Systems Engineering, pp. 74–89.

Fischer, S., Urbanke, P., Ramler, R., Steidl, M., Felderer, M., 2024. An overview of microservice-based systems used for evaluation in testing and monitoring: a systematic mapping study. In: 2024 IEEE/ACM International Conference on Automation of Software Test (AST), pp. 182–192.

Foundation, A.S., 2025a. Apache Skywalking: Application Performance Monitoring and Observability Platform. Accessed 27 May 2025. https://skywalking.apache.org/.

Foundation, C.N.C., 2025b. Kubernetes: Open-Source Container Orchestration Platform. Accessed 27 May 2025. https://kubernetes.io/.

Giamattei, L., Guerriero, A., Pietrantuono, R., Russo, S., 2024. Automated functional and robustness testing of microservice architectures. J. Syst. Softw. 207, 111857. https://doi.org/10.1016/j.jss.2023.111857.

Group, A., 2025. Nacos: Dynamic Service Discovery, Configuration, and Service Management Platform. Accessed 27 May 2025. https://nacos.io/en/.

Group, C.S., 2018. CUHK-SE-Group Train-Ticket System. Commit hash: be0f7a85, Date: 2025-03-22. Accessed 27 May 2025. https://github.com/CUHK-SE-Group/train-ticket/.

Gu, W., Gu, J., Liu, J., Chen, Z., Zhang, J., Kuang, J., Feng, C., Yang, Y., Lyu, M.R., 2025. ADAMAS: Adaptive domain-aware performance anomaly detection in cloud service systems. In: Proceedings of the International Conference on Software Engineering (ICSE), pp. 621.

Holt, M., 2025. Caddy: The Ultimate Server with Automatic HTTPS. Accessed 27 May 2025. https://caddyserver.com/.

L'Aquila, S. a. U.o., 2018. SEALABQualitygroup Train-Ticket System. Commit hash: 3ab49b0, Date: 2025-03-06. Accessed 27 May 2025. https://github.com/SEALABQualityGroup/train-ticket.

Larrucea, X., Santamaria, I., Colomo-Palacios, R., Ebert, C., 2018. Microservices. IEEE Softw. 35, 96–100.

Li, S., Zhang, H., Jia, Z., Zhong, C., Zhang, C., Shan, Z., Shen, J., Babar, M.A., 2021. Understanding and addressing quality attributes of microservices architecture: a systematic literature review. Inf. Softw. Technol. 131, 106449.

Liang, P., Xun, Y., Cai, J., Yang, H., 2026. Autoscaling of microservice resources based on dense connectivity spatio-temporal GNN and q-learning. Fut. Gener. Comput. Syst. 174, 107909.

Liao, L., Eismann, S., Li, H., Bezemer, C.-P., Costa, D.E., van Hoorn, A., Shang, W., 2025. Early detection of performance regressions by bridging local performance data and architectural models. In: 2025 IEEE/ACM 4th International Conference on Software Engineering (ICSE). IEEE Computer Society, Los Alamitos, CA, USA, pp. 317–329. https://doi.org/10.1109/ICSE55347.2025.00026.

Liu, L., Tu, Z., He, X., Xu, X., Wang, Z., 2021. An empirical study on underlying correlations between runtime performance deficiencies and "bad smells" of microservice systems. In: Proceedings of the International Conference on Web Services (ICWS), pp. 751–757.

MacCormack, A., Rusnak, J., Baldwin, C.Y., 2006. Exploring the structure of complex software designs: an empirical study of open source and proprietary code. Manage. Sci. 52, 1015–1030.

Mo, R., Cai, W.S.Y., Ramaswamy, S., Kazman, R., Naedele, M., 2018. Experiences applying automated architecture analysis tool suites. In: Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering, pp. 779–789.

Mo, R., Cai, Y., Kazman, R., Xiao, L., Feng, Q., 2016. Decoupling level: a new metric for architectural maintenance complexity. In: Proc. 38rd C-ICSE, p. 499–510.

Mo, R., Cai, Y., Xiao, L., Kazman, R., Feng, Q., 2021. Architecture anti-patterns: automatically detectable violations of design principles. IEEE Trans. Softw. Eng. 47.

MongoDB, I., 2025. MongoDB: The World's Leading Modern Database. Accessed 27 May 2025. https://www.mongodb.com/.

Nayebi, M., Cai, Y., Kazman, R., Ruhe, G., Feng, Q., Carlson, C., Chew, F., 2019. A longitudinal study of identifying and paying down architecture debt. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 171–180. https://doi.org/10.1109/ICSE-SEIP.2019.00026.

NGINX, I., 2025. NGINX: The High-Performance Web Server and Reverse Proxy. Accessed 27 May 2025. https://nginx.org/.

OpenZipkin, 2025. Zipkin: A Distributed Tracing System. Accessed 27 May 2025. https://zipkin.io/.

Oracle, 2025. jdeps: Java Class Dependency Analyzer. Accessed 27 May 2025. https://docs.oracle.com/en/java/javase/11/tools/jdeps.html.

Quattrocchi, G., Incerto, E., Pinciroli, R., Trubiani, C., Baresi, L., 2024. Autoscaling solutions for cloud applications under dynamic workloads. IEEE Trans. Serv. Comput. 17, 804–820.

Raychev, N., 2020. Test automation in microservice architecture. IEEE Spectr. 8, 15.

Rodrigues, H., Silva, A.R., Avritzer, A., 2025. Assessment of performance and its scalability in microservice architectures: systematic literature review. J. Syst. Softw. 112500.

Rud, A., 2019. Why and How netflix, Amazon, and Uber Migrated to Microservices: Learn from their Experience. Accessed 27 May 2025. https://web.archive.org/web/20230104141759/.

Team, S., 2025. Spring Framework. Accessed 27 May 2025. https://spring.io/projects/spring-framework.

Trubiani, C., Ghabi, A., Egyed, A., 2017. Exploiting traceability uncertainty between software architectural models and extra-functional results. J. Syst. Softw. 125, 15–34.

Trubiani, C., Pinciroli, R., Biaggi, A., Fontana, F.A., 2023. Automated detection of software performance antipatterns in java-based applications. IEEE Trans. Softw. Eng. 49, 2873–2891.

University, S.E.L. of, 2021. Fudan v0.2.0 Train-ticket System. Commit hash: e5ae0c3a, Date: 2025-03-12. Accessed: 2025-05-27. https://github.com/FudanSELab/train-ticket/releases/tag/v0.2.0.

University, S.E.L. o.F., 2022. Fudan v1.0.0 Train-Ticket System. Commit hash: fa8d9ef, Date: 2025-05-02. Accessed: 2025-05-27. https://github.com/FudanSELab/train-ticket/releases/tag/v1.0.0.

University, S. g. a.C., 2019. Deathstarbench. Last Accessed 27 May 2025. https://github.com/delimitrou/DeathStarBench.

Wang, Y., Cheng, L., Sun, X., 2019. Design and research of microservice application automation testing framework. In: International Conference on Information Technology and Computer Application (ITCA), pp. 257–260. https://doi.org/10.1109/ITCA49981.2019.00063.

Wohlin, C., 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering. Association for Computing Machinery, New York, NY, USA, pp. 1–10. EASE '14, https://doi.org/10.1145/2601248.2601268.

Wu, W., Cai, Y., Kazman, R., Mo, R., Liu, Z., Chen, R., Ge, Y., Liu, W., Zhang, J., 2018. Software architecture measurement—experiences from a multinational company. In: Cuesta, C.E., Garlan, D., Pérez, J. (Eds.), Software Architecture. Springer International Publishing, pp. 303–319.

Zhou, X., Peng, X., Xie, T., Sun, J., Xu, C., Ji, C., Zhao, W., 2018. Poster: benchmarking microservice systems for software engineering research. In: 2018 IEEE/ACM 40th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). IEEE Computer Society, Los Alamitos, CA, USA, pp. 323–324. https://doi.ieeecomputersociety.org/.